

File di configurazione utenti

/etc/skel — scheletro della home. Quando `useradd -m` crea la home di un nuovo utente, copia al suo interno tutto il contenuto di questa directory. È il punto in cui aggiungere file che ogni nuovo utente deve avere per default (`.bashrc` preconfigurato, `.vimrc`, ecc.).

/etc/default/useradd — valori di default per `useradd` quando non vengono passate opzioni esplicite. Qui è definita, ad esempio, la shell di default (spesso `sh`, ecco perché bisogna sempre passare `-s /bin/bash` a mano).

/etc/login.defs — configurazione più bassa: policy globali di sistema. Contiene le regole su scadenza password, intervallo UID per gli utenti fisici (1000–60000), `umask` di default.

Directory collaborativa con SGID

Perché i nuovi utenti hanno UID 1004 e 1005?

Gli UID vengono assegnati in incremento e **non si riutilizzano automaticamente**. Se hai creato e poi eliminato utenti con UID 1001, 1002, 1003, quei numeri rimangono “usati” nella storia del sistema. I prossimi utenti creati prendono il primo UID libero a partire dall’ultimo assegnato. È un comportamento intenzionale: evita che un nuovo utente erediti per sbaglio i permessi di uno vecchio su file rimasti nel filesystem.

Sotto quale utente viene creata la directory?

```
sudo mkdir /home/programmatori
```

`sudo` esegue come `root`. La directory viene quindi creata con `root` come proprietario. Non è vagrant — è `root`. Puoi verificarlo con `ls -la /home/` che mostrerà `root root` come owner:group prima del passo successivo.

Cosa fa `chgrp programmatori /home/programmatori`?

Cambia il **gruppo proprietario** della directory. Dopo questo comando la directory ha `root` come utente proprietario e `programmatori` come gruppo proprietario. È questo che permette ai membri del gruppo di entrarci — i permessi del blocco “group” si applicano a tutti i membri di `programmatori`.

Cos'è .bashrc?

.bashrc è uno script che Bash esegue automaticamente ogni volta che apri una nuova sessione interattiva (ogni su - nome, ogni nuovo terminale). Contiene configurazioni personali dell'utente: alias, variabili d'ambiente, e — nel nostro caso — la umask. Il punto iniziale del nome (.bashrc) lo rende un file nascosto — non compare in ls, solo in ls -a.

Cos'è umask e perché serve?

Quando crei un file, Linux parte da un massimo teorico e **sottrae** la umask per calcolare i permessi effettivi:

- File: massimo 666 (rw-rw-rw-) – umask 0002 = 664 (rw-rw-r-)
- Directory: massimo 777 (rwxrwxrwx) – umask 0002 = 775 (rwxrwxr-x)

Con la umask di default 0022 i file vengono creati con permessi 644 (rw-r-r-): il gruppo può solo leggere, non scrivere. Per la collaborazione serve che il gruppo possa scrivere, quindi si usa 0002 che rimuove solo la scrittura agli "others".

Senza questa modifica, maria creerebbe un file con permessi 644 e piero non potrebbe modificarlo anche essendo nel gruppo programmatori.

Cosa fa sudo tee -a?

Il problema: non puoi fare `sudo echo "... " >> /home/maria/.bashrc` perché il redirect >> viene gestito dalla shell dell'utente corrente (vagrant), non da sudo — quindi non ha i permessi per scrivere nella home di maria.

La soluzione è tee: è un comando che legge da stdin e scrive su file. Passandogli sudo, è tee (con privilegi root) a aprire il file, non la shell.

```
echo "umask 0002" | sudo tee -a /home/maria/.bashrc
#
#
#
#
```

*~~~~~
tee apre il file come root
~~~~~  
la stringa arriva via pipe a tee*

-a sta per **append** — aggiunge in fondo senza sovrascrivere il contenuto esistente del .bashrc.

## Perché chmod 02770 ha quattro cifre?

I permessi in ottale sono sempre **quattro cifre**, non tre. La prima cifra (quella davanti) codifica i **bit speciali**:

| Prima cifra | Effetto                                                       |
|-------------|---------------------------------------------------------------|
| 0           | Nessun bit speciale (default implicito nelle forme a 3 cifre) |
| 1           | Sticky bit                                                    |
| 2           | <b>SGID</b>                                                   |
| 4           | SUID                                                          |

`02770` = SGID (2) + `rw-rwx---` (770).

Quando scrivi `chmod 770` stai implicitamente scrivendo `chmod 0770` — la prima cifra zero viene omessa perché nessun bit speciale è attivo.

### Perché il SGID è indispensabile?

Senza SGID sulla directory, quando maria crea un file il gruppo proprietario del file è maria (il suo gruppo primario). Piero non è nel gruppo maria — cadrebbe nel blocco “others” che ha ---. Non potrebbe né leggere né scrivere.

Con SGID attivo sulla directory, il file viene creato con gruppo programmatori invece del gruppo primario di maria. Piero è in programmatori e quindi può accedervi.

Senza SGID: maria crea file → gruppo = maria → piero: others → ---  
 Con SGID: maria crea file → gruppo = programmatori → piero: group → rw-

### I quattro meccanismi che cooperano

| Meccanismo                                       | Perché serve                                                               |
|--------------------------------------------------|----------------------------------------------------------------------------|
| Gruppo programmatori                             | Definisce chi appartiene alla “squadra”                                    |
| <code>chmod 02770</code>                         | SGID attivo + solo owner e group possono accedere alla directory           |
| SGID sulla directory                             | I file creati dentro ereditano il gruppo programmatori automaticamente     |
| <code>umask 0002</code> nel <code>.bashrc</code> | I nuovi file hanno permessi 664 — il gruppo può scrivere, non solo leggere |

Togliere uno solo di questi quattro elementi rompe la collaborazione.

### Risultato del test

```
-rw-rw-r-- 1 maria programmatori ... file_di_maria.txt
```

Il gruppo programmatori (non maria) comparso automaticamente sul file è la prova che SGID funziona. Piero può leggere e scrivere perché: 1. è nel gruppo programmatori 2. i permessi del blocco group sono rw- 3. quei permessi si applicano a lui perché il gruppo del file è programmatori